

中山大学 计算机学院本科生实验报告

(2025/2026学年)

课程名称	并行程序设计与算法 (实验)	批改人	
实验	Lab 5 - 基于OpenMP的并行矩阵乘法与自定义并行模式	专业 (方向)	计算机科学与技术 (人工智能)
学号	23336103	姓名	雷颜玮
Email	leiyanyanwei2005@163.com	完成日期	2026年5月2日

1. 实验要求与目的

- 基于OpenMP实现通用矩阵乘法优化**: 使用OpenMP实现不同线程规模下 (1~8) 的GEMM (General Matrix Multiply), 测定多阶矩阵规模。并针对 `schedule(static, 1)` 和 `schedule(dynamic, 1)` 等不同的任务调度机制进行性能比较。
- 基于Pthreads构造并打包 `parallel_for` 动态链接库**: 通过模仿 OpenMP 中 `#pragma omp parallel for` 的行为, 手工使用 Pthreads 包装并实现循环分解、分配和线程启动逻辑, 将其编译为 `.so` 动态链接库, 并在此基础上改造 GEMM 进行正确性与有效性验证。

2. 任务一与二: 基于OpenMP的GEMM及调度方式比较

2.1 OpenMP 并行逻辑与代码实现

对最外层循环 `for (int i = 0; i < M; ++i)` 加 `#pragma omp parallel for`, 由于矩阵乘法的不同行之间没有依赖, 故不同线程各自负责一部分行。

```
// 以 Dynamic 调度为例
#pragma omp parallel for schedule(dynamic, 1)
for (int i = 0; i < M; ++i) {
    for (int k = 0; k < K; ++k) {
        float a_ik = A[i * K + k]; // 循环次序优化 (i->k->j) 以利用空间局部性缓存
        for (int j = 0; j < N; ++j) {
            C[i * N + j] += a_ik * B[k * N + j];
        }
    }
}
```

2.2 多种调度方式下的测试比较

通过自动化脚本将 $M=N=K$ 从 512 变动到 2048, 线程 1 到 8。以下是真实跑分的汇总表格:

Size	Threads	Schedule	Time (s)
512	1	Default	0.0129
512	4	Default	0.0034

Size	Threads	Schedule	Time (s)
512	8	Default	0.0025
1024	1	Default	0.0916
1024	4	Default	0.0287
1024	8	Default	0.0186
2048	1	Default	1.0610
2048	4	Default	0.2691
2048	8	Default	0.1888
2048	8	Static(1)	0.1937
2048	8	Dynamic(1)	0.1636

性能分析与调度策略讨论：

- 1. 明显见到 Omp 极大地缩短了耗时，例如在 2048 规模下，从完全单线程的 1.06s 缩减至 4线程的 0.269s 左右，得到优秀的加速比。
- 2. **调度策略对比**：在 2048 大小的矩阵乘法中，每次 i 行的迭代需要经过完整的运算。相比于块分配（Default, 0.1888s），细粒度交错派发任务的 Static(1) 在 8 线程下可能由于破坏了基于行内存邻近的 Cache 预取导致速度没有提升。而 Dynamic(1) 由于开销动态调度（0.1636s），跑出了较好成绩，可能是系统本身核心频率波动或分配差异导致，但整体差距未拉开，这也说明在所有行负载皆均匀的情况下，各种调度本质上并无统治性优势，有时仅仅引入了更重的线程同步开销。

3. 任务三：自定义 parallel_for 动态链接库

通过 Pthreads 实现一套属于自己的“轮子”代替 OpenMP 进行 for 循环并行派发。

3.1 parallel_for 接口设计与打包动态链接库

在 lib/libparallelfor.so 中实现了如下函数原型，依据参数中的 start，end 和 inc 在内部利用商余数分配的算法，计算每个 Pthread 所应承担的循环区间。被创建的子线程内部执行循环体并调用外部传进的代码回调（functor）。

```
void parallel_for(int start, int end, int inc,
                 void *(*functor)(int, void*), void *arg, int num_threads) {
    // 省略线程合法性检查...
    int total_iterations = (end - start + inc - 1) / inc;
    int iter_per_thread = total_iterations / num_threads;
    int remainder = total_iterations % num_threads;

    // ... 对各个线程分发区间块
    for (int i = 0; i < num_threads; ++i) {
        int iters = iter_per_thread + (i < remainder ? 1 : 0);
        t_data[i].start = start + current_iter * inc;
        t_data[i].end = t_data[i].start + iters * inc;
        // 调用 pthread_create 拉起 worker 执行区间内的 for (...; i += inc) {
        functor(i); }
```

```

        pthread_create(&threads[i], nullptr, worker_thread, &t_data[i]);
        current_iter += iters;
    }
    // join ...
}

```

3.2 矩阵乘法接入自定义 parallel_for 代码

按照课设要求，我们将 GEMM 的具体任务放入 `functor` 中，并通过 `functor_args` 这个 struct 打包矩阵的原始指针与参数。

```

struct functor_args { float *A, *B, *C; int M, N, K; };

void* gemm_functor(int idx, void* args) {
    functor_args* data = (functor_args*)args;
    int i = idx;
    // idx 充当被分配好的列项：代替了 i
    for (int k = 0; k < data->K; ++k) {
        float a_ik = data->A[i * data->K + k];
        for (int j = 0; j < data->N; ++j) {
            data->C[i * data->N + j] += a_ik * data->B[k * data->N + j];
        }
    }
    return nullptr;
}

// 主函数中直接调用（使用 4 线程）
parallel_for(0, M, 1, gemm_functor, (void*)&args, 4);

```

3.3 自定义库的测试结果

我们同样使用该套自己封装的 Pthread 实现对各个维度的矩阵计算真实跑分：

Size	Threads	Time (s)
512	1	0.0132
512	2	0.0061
512	4	0.0031
512	8	0.0021
1024	1	0.0899
1024	2	0.0469
1024	4	0.0240
1024	8	0.0149
2048	1	1.0413
2048	2	0.5681
2048	4	0.2471

Size	Threads	Time (s)
2048	8	0.1375

性能分析：
如真实环境数据所示，这套通过块分配（Block Scheduling）理念写成的自定义接口表现**甚至不亚于乃至略强于 OpenMP 的默认实现**。在 2048 规模、8线程的情况下跑出了 0.1375 秒的极佳成绩（甚至打败了 OpenMP Default 的 0.1888s），不仅说明 `pthread_create/join` 以静态切分对均衡负载而言足矣，且证明对于矩阵这种全均载任务，开局静态的一次性大局分配反而由于没有任何中间锁的管理开销，达到了最高的加速扩展性（加速比 ≈ 7.57 ）。该代码完美接入并跑通了 `.so` 动态库的调用模式。